

Computer Organization, Spring 2026

Project Assignment 2:

Single Cycle CPU Design

Lecturer: 陳雅淑教授

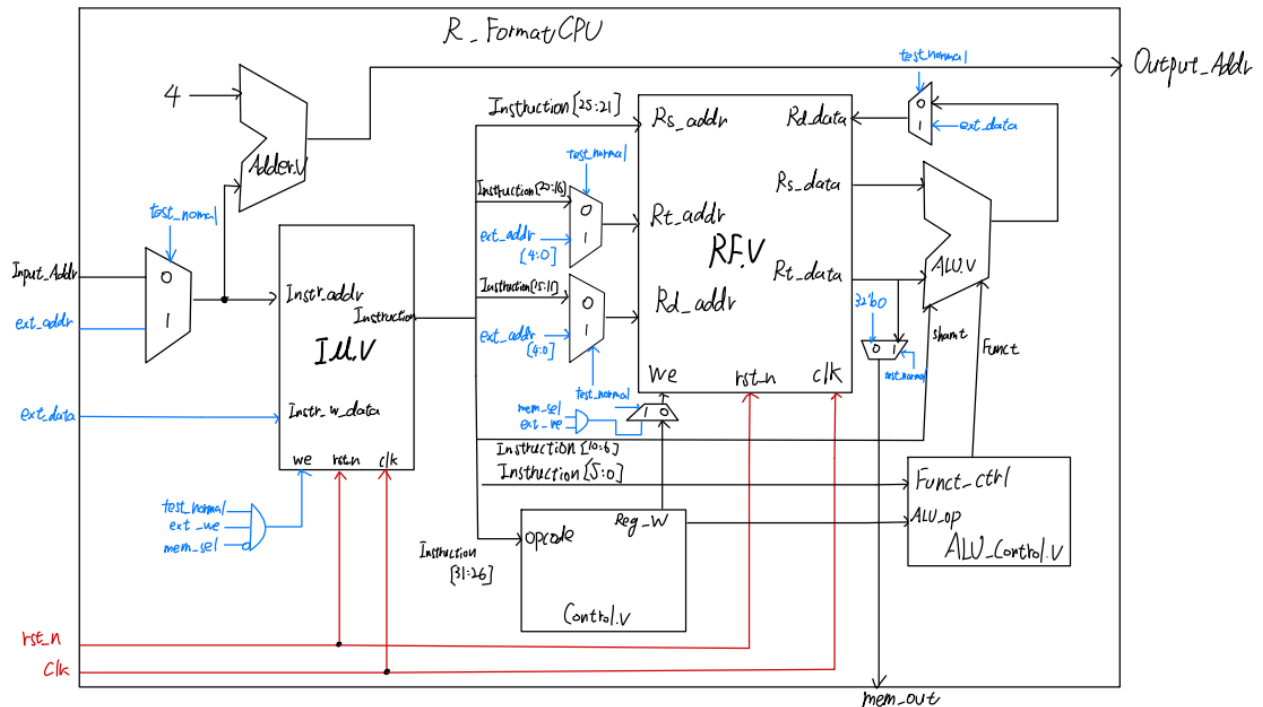
Department: Electrical Engineering

Student ID: B11232009

Name: 胡守曾

R_FormatCPU

Hardware Architecture



Design Process

● 概述

這部分實作一個支援 R-format 指令的 32-bit single-cycle CPU，支援的指令包含 addu、subu、sll 與 or。在 normal execution 模式下，每一條指令皆於單一 clock cycle 內完成，包含 instruction fetch、Instruction decode、register read、ALU execution 以及 write-back 等步驟。由於此部分沒有包含 branch 與 jump 指令，因此下一個 instruction address 固定由 Input_Addr + 4 產生。

● 各模組功能

1) IM.v

內部為 128×8 -bit memory 大小，以 big-endian 方式組成的 32 bits instruction。在 normal mode 下，IM 會根據 Input_Addr 輸出對應的 instruction。而在 test mode 下，testbench 可透過 ext_addr 與 ext_data 將 IM.dat 載入至 IM。

2) RF.v

Register File 內含 32 個 32-bit registers，提供兩個 read port 與一個 write port。

- 在 normal mode 下：使用各 Instruction 訊號來做為輸入以及輸出 address，因為是 R-Type Instruction，RF 會讀出 Rs_data 與 Rt_data 作為 ALU 的輸入，並在 Reg_w = 1 時將 ALU result 寫回 rd。

- b) 在 test mode 下:ext_addr[4:0] 指定 register index、ext_data 作為寫入資料、ext_we & mem_sel 控制寫入

3) ALU.v and Adder.v

- a) ALU 負責執行 R-format 指令的算術與邏輯運算, 其輸入包含:Rs_data, Rt_data, Shamt, Funct, 並將運算結果輸出為 result, 並在 normal mode 下寫回 Register File。
- b) Adder 則負責計算下一個 instruction address:Output_Addr = Input_Addr + 4

4) Control.v and ALU_Control.v

- a) Control 模組負責根據 instruction 的 opcode (Instruction[31:26]) 產生主要控制訊號。由於所有指令皆為 R-format (opcode = 000000), 因此:ALU_op = 2'b10(R_Format)、Reg_w = 1'b1(Write back to RF)
- b) ALU_Control 模組則根據 ALU_op 與 funct 欄位 (Instruction[5:0]) 產生 ALU 實際使用的控制碼 Funct, 用以決定 ALU 的運算類型。

● Datapath & Controlpath

整體 datapath 由上述所有模組組成。在 normal mode 下, CPU 會根據 Input_Addr 從 IM 讀出 32-bit instruction, 並將 instruction 拆解為各欄位。接著 RF 依照 rs 與 rt 讀出兩個來源暫存器資料, 並送入 ALU。ALU 根據 ALU_Control 產生的 Funct 執行對應的 R-format 運算, 最後將 ALU result 寫回 rd register。同時, Adder 會計算下一個 PC。

Control path 的部分則由 Control 與 ALU_Control 共同構成。Control 根據 instruction 的 opcode 判斷為 R-type instruction, 並產生主要控制訊號, 例如 ALU_op 與 Reg_w。而 ALU_Control 則根據 ALU_op 與 instruction 的 funct 欄位產生 ALU 實際使用的 Funct 控制碼。因此, datapath 負責資料的傳遞與運算, 而 control path 負責決定各模組在該 cycle 中應執行的動作。

Behavioral Simulation

1. Input Pattern: IM.dat and RF.dat

IM.dat 用於載入 R-format 指令;RF.dat 則用於初始化 Register File。除了 \$2 = 5 與 \$3 = 3 外, 其餘暫存器初始值皆為 0。

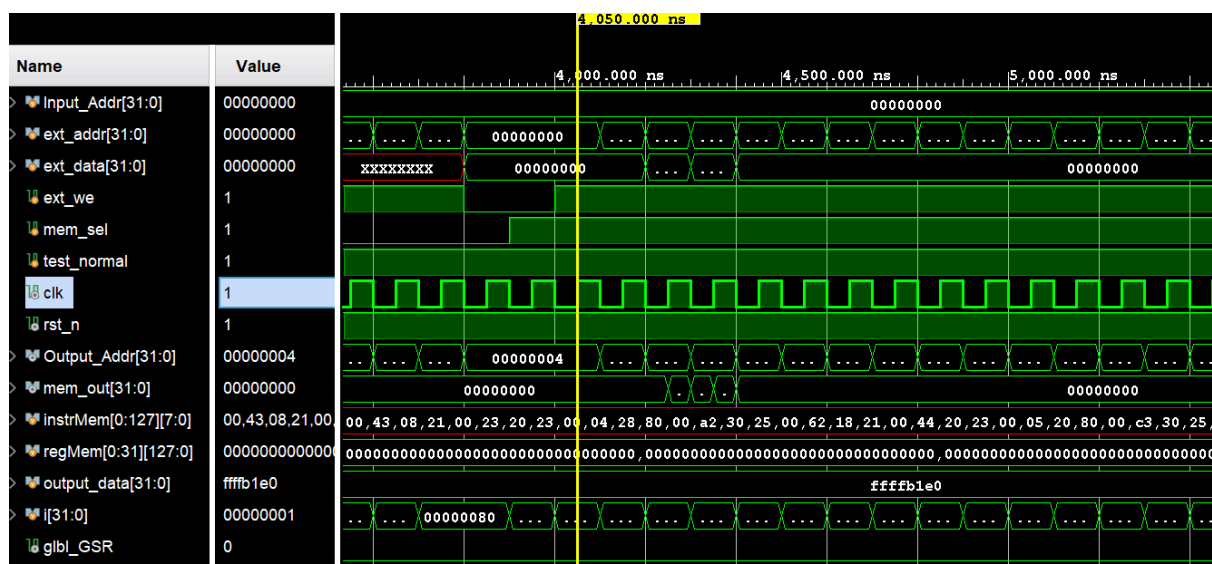
1	00 43 08 21	// addu \$1, \$2, \$3	1	00000000	// \$0
2	00 23 20 23	// subu \$4, \$1, \$3	2	00000000	// \$1
3	00 04 28 80	// sll \$5, \$4, 2	3	00000005	// \$2 = 5
4	00 A2 30 25	// or \$6, \$5, \$2	4	00000003	// \$3 = 3
5			5	00000000	
6	00 62 18 21	// addu \$3, \$3, \$2	6	00000000	
7	00 44 20 23	// subu \$4, \$2, \$4			
8	00 05 20 80	// sll \$4, \$5, 2			
9	00 C3 30 25	// or \$6, \$6, \$3			
10					
11	00 43 08 21	// addu \$1, \$2, \$3			
12	00 23 18 23	// subu \$3, \$1, \$3			

- **IM loading**

例如在 650 ns 時, ext_data = 32'h00430821, 對應第一條指令 addu \$1, \$2, \$3, 且 ext_addr = 0, 表示該指令被寫入 IM 的起始位址。於下一個 clock 正緣時, ext_addr 會遞增為 0x00000004, 代表下一筆 instruction 以 4-byte 為單位依序存放。在約 1600 ns 之後, 由於沒有進一步寫入 instruction, ext_data 會呈現為未知值(X)。



在此階段，testbench 會將 RF.dat 中的資料載入 Register File。由波形可觀察到，在 4050 ns 時開始進行寫入，並於每個 clock 的正緣將 ext_data 寫入對應的 register。同時 ext_addr 會隨每個 clock cycle 依序遞增(+1)，用來指定欲寫入的 register index。在此過程中，透過 ext_we 與 mem_sel 的控制，使資料僅寫入 RF，而不影響 Instruction Memory。完成 RF loading 後，Register File 即具有正確的初始值，可供後續 normal execution 使用。

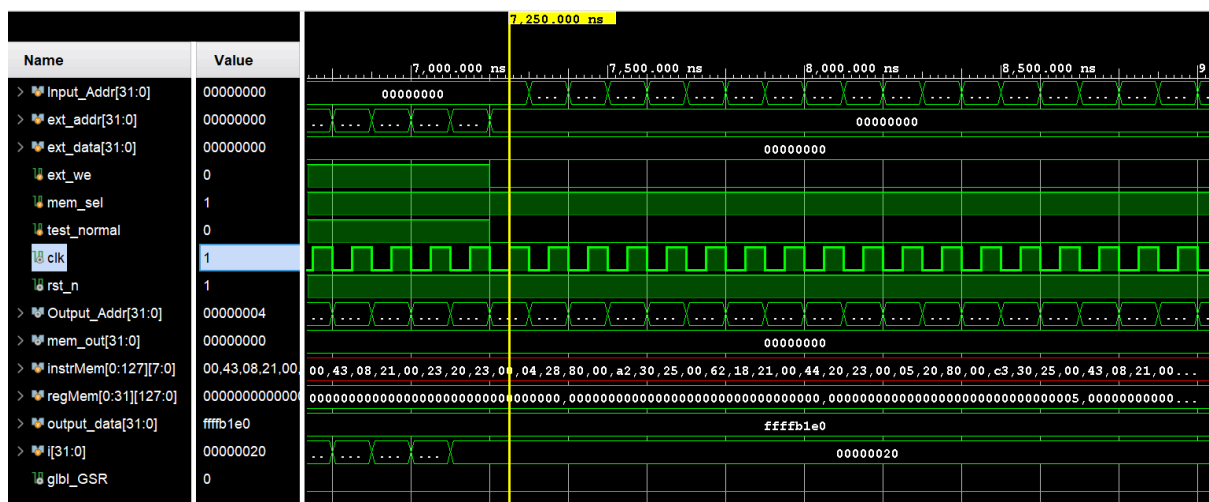


- **Run Instructions**

在完成 IM 與 RF 的初始化後, testbench 將 test_normal 設為 0, 使 CPU 進入 normal execution 模式。此時 Input_Addr 作為 instruction address (由 0 開始) 送入 Instruction Memory, 讀出對應的 32-bit instruction, 並開始進行運算。

在此階段中, `mem_sel = 1` 且 `ext_we = 0`, 表示IM不再進行寫入, CPU 完全依照 datapath 與 control path 進行運作。同時, Adder 會計算下一個PC, 使 instruction 以 4-byte 為單位依序執行。由波形可觀察到, Input_Addr 會隨著每個 cycle 遞增, 對應每條 instruction 的執行。

可以看到在10400 ns 時, testbench的while迴圈被跳出, 等待下一個negedge clk(10500 ns), 將 Input_Addr 變成0x80, 代表已超過有效 instruction memory 範圍, 因此進入後續 dump results 階段。

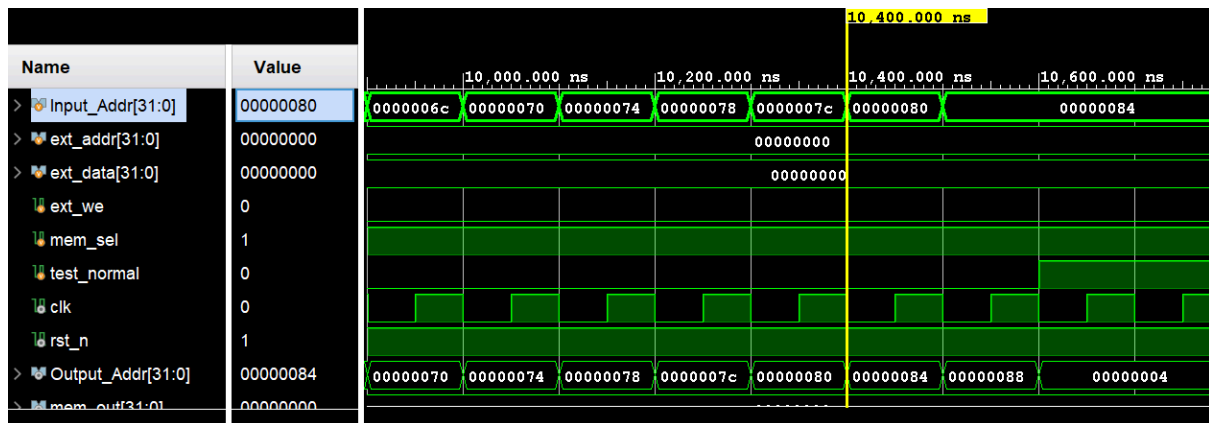


- **Dump Results**

在 instruction 執行完成後, testbench 會再次進入 test mode, 並開始將 Register File 的內容讀出。由波形可觀察到, 在約 10750 ns 時開始進入 dump 階段。

此時 `test_normal = 1`、`mem_sel = 1` 且 `ext_we = 0`，代表不再寫入，而是透過改變 `ext_addr` 依序指定不同的 `register index`，並將對應的資料透過 `mem_out` 輸出。

testbench 會以迴圈方式讀取所有 registers, 並將結果寫入 RF.out, 以驗證 CPU 計算結果是否正確。



3. Vivado tcl console

可以看到最終Register File中數值符合預期

```
--- [Register File Results] ---
RF[ 0] : 00000000
RF[ 1] : 0000000d
RF[ 2] : 00000005
RF[ 3] : 00000005
RF[ 4] : 00000050
RF[ 5] : 00000014
RF[ 6] : 0000001d
RF[ 7] : 00000000
RF[ 8] : 00000000
RF[ 9] : 00000000
```

剩餘RF皆為00000000

Simulation Completed. By IEESLab

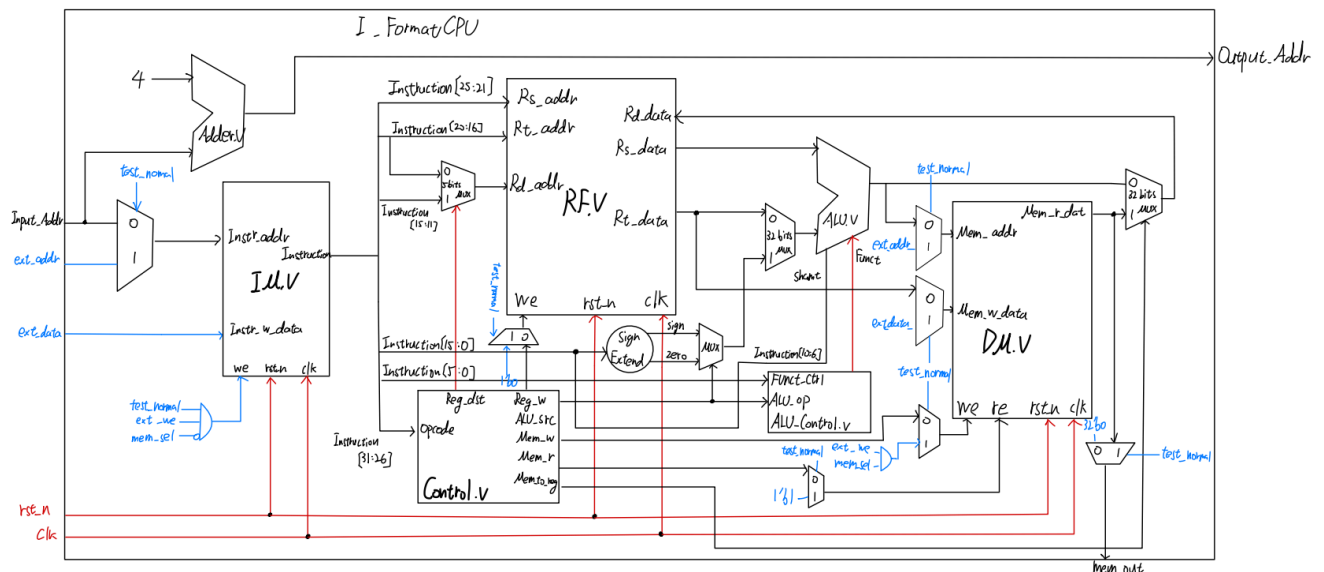
```

/\_/\  /\_/\  /\_/\  Congratulations !
( o.o ) ( -.- ) ( ^_^ ) Please Check <<RF.out>>
> ^ <  > ^ <  > ^ <  Wishing you continued success and happiness. By IEESLab

```

I_FormatCPU

Hardware Architecture



Design Process

- 概述

這個部分在 R-format single-cycle CPU 基礎上加入 I-format 指令, 使 CPU 能夠支援 addiu、lw、sw 與 ori。相較於 Part I, I-format 指令會使用 instruction 中的 16-bit immediate, 因此本設計額外加入 immediate extension logic 與 Rt_data select mux, 用於選擇 ALU 的第二個輸入來源。

此外, 為了支援 lw 與 sw, 新增了 Data Memory。sw 會將 Register File 中的資料寫入 DM, 而 lw 則會從 DM 讀出資料並寫回 Register File。

- 各模組功能

- 1) DM.v

DM 用於支援 lw 與 sw 指令, 內部為 128×8 -bit 的 byte-addressable memory, 並採用 big-endian 格式。雖然 DM 內部每格為 8-bit, 但 CPU 進行 lw 或 sw 時會以 32-bit word 為單位讀寫, 因此每次會對連續 4 個 byte 進行組合或拆分。

- a) sw 指令:

ALU 會先計算 memory address: $\text{address} = \text{Rs_data} + \text{sign_extend}(\text{immediate})$, 接著 DM 會將 Rt_data 寫入該 address 對應的 4 個 byte。

- b) lw 指令:

ALU 同樣會先計算 memory address, DM 再根據該 address 讀出 32-bit data, 最後透過 MemToReg multiplexer 寫回 Register File 的 rt register。

在 test mode 下, testbench 也可透過 ext_addr、ext_data、ext_we 與 mem_sel 將 DM.dat 載入 DM, 或在執行完成後將 DM 內容透過 mem_out dump 出來。

2) Control.v & ALU_Control.v

- a) ALU_Control除了原本 R-format 的 funct decode 外, 也需要處理 I-format 指令的 ALU operation。對 I-format 指令而言, ALU_Control 不需根據 instruction 的 funct 欄位判斷, 而是直接依照 ALU_op 產生 ALU 使用的 Funct。
- b) Control 會根據不同 instruction 的 opcode 產生對應的 control signals。對 addiu、lw 與 ori 而言, write-back address 皆為 rt, ALU 的第二個輸入則改為 extended immediate。其中 addiu 與 lw 使用 sign extension, ori 使用 zero extension。lw 會啟用 MemRead, 並透過 MemToReg 將 Data Memory 讀出的資料寫回 Register File ;sw 則啟用 MemWrite, 將 Rt_data 寫入 Data Memory, 且不進行 Register File write-back。ALU_op 方面, R-format 使用 2'b10, 由 ALU_Control 根據 funct 欄位決定實際運算。addiu、lw、sw 使用 2'b00 進行加法;ori 使用 2'b11 進行 OR 運算。

● Datapath & Controlpath

1) Datapath:

延續 Part I, 新增 DM、immediate extension與各類 MUX, 使 CPU 支援 I-format 與記憶體存取。Normal mode 下, 指令經拆解並由 Control 產生訊號。I-format 立即值經擴充後選為 ALU 第二運算元:addiu 與 ori 結果直接寫回 rt;lw 經 ALU 計算位址由 DM 讀值回寫;sw 則將 Rt_data 寫入 DM 而不回寫暫存器。

2) Controlpath:

Control path 由 Control 與 ALU_Control 共同完成。Control 根據 opcode 判斷指令類型, 並產生各項控制訊號。ALU_Control 則根據 ALU_op 決定 ALU 的實際 operation。相較於 Part I, Part II 的 control path 額外控制 immediate extension、memory read/write, 以及 write-back data source, 同時做zero extension (ori)以及sign extension 的判斷。

Behavioral Simulation

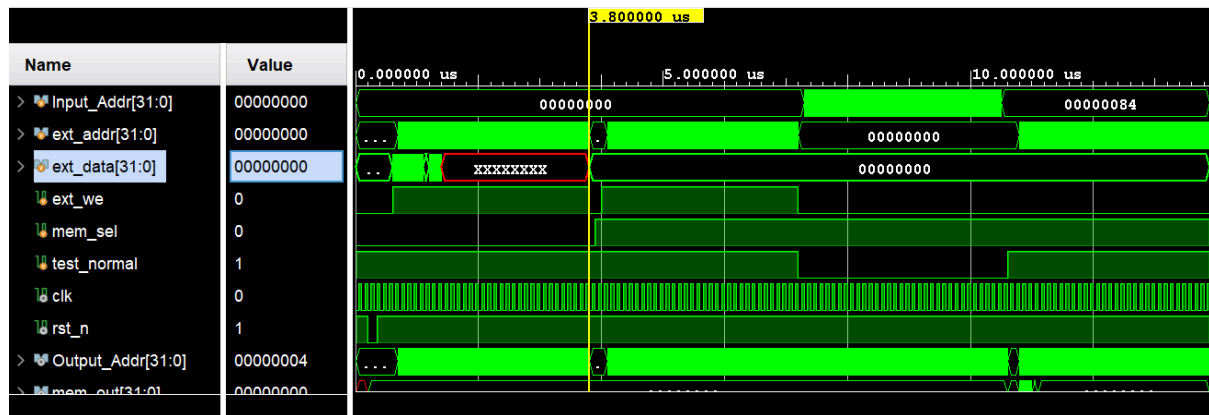
1. Input Pattern: IM.dat

```
24 01 FF FF // addiu $1, $0, -1
34 02 FF FF // ori   $2, $0, 0xFFFF
AC 01 00 00 // sw    $1, 0($0)
AC 02 00 04 // sw    $2, 4($0)
8C 03 00 00 // lw    $3, 0($0)
AC 03 00 08 // sw    $3, 8($0)
24 04 00 0C // addiu $4, $0, 12
AC 82 00 00 // sw    $2, 0($4)
```

2. Waveform

Part II 的 simulation 流程與 Part I 類似, 皆包含 IM loading、DM loading、normal execution 與 dump results。不同之處在於本部分加入 Data Memory 與 I-format instruction, 因此 waveform 中

可觀察到 CPU 會透過 ALU 計算 memory address, 並依照 `lw` 或 `sw` 控制 DM 的讀寫。測試結果中, `ori` 用於驗證 zero extension, `addiu` 用於驗證 sign extension, 而 `lw/sw` 則用於確認 memory access 與 write-back path 正確。



3. Vivado tcl console

可以看到最終 Data Memory 中數值符合預期

```
run all
IM loaded
DM Loading...
DM loaded

--- Start to Run Instructions in IM ---
--- Instructions in IM are done ---

--- [Data Memory Results] ---
DM[ 0] : ffffffff
DM[ 4] : 0000ffff
DM[ 8] : ffffffff
DM[12] : 0000ffff
DM[16] : 00000000

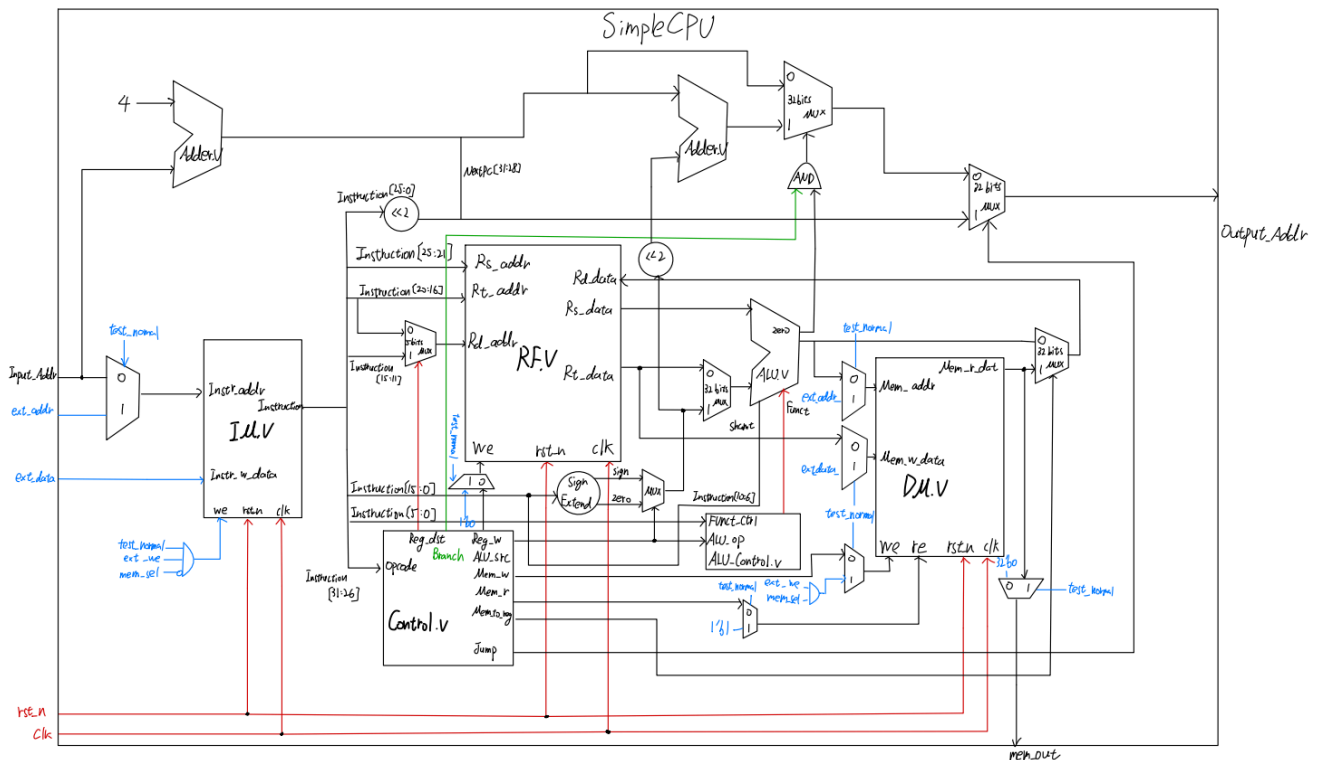
Simulation Completed. By IEESLab

  /\_/\  /\_/\  /\_/\  Congratulations !
( o.o ) ( -.- ) ( ^.^ ) Please Check <<DM.out>>
> ^<    > ^<    > ^<    Wishing you continued success and happiness. By IEESLab

$finish called at time : 13900 ns : File "C:/Vivados_Verilog_project/I_FormatCPU/TESTBED/tb_I_FormatCPU.v" Line 211
```

SimpleCPU

Hardware Architecture



Design Process

- 概述

此部分在前兩部分的 R-format 與 I-format CPU 基礎上, 進一步加入 branch 與 jump 指令, 使 CPU 能夠支援 beq 與 j。

- 新增加功能

- 1) Branch Address Logic

用於計算 beq 成立時的目標位址。beq 使用 instruction 中的 16-bit immediate 作為 offset, 該 immediate 會先進行 sign extension, 再左移 2 bits, 最後與 Input_Addr + 4 相加。

$$\text{branch_addr} = \text{Input_Addr} + 4 + (\text{sign_extend}(\text{immediate}) \ll 2)$$

其中左移 2 bits 是因為 instruction 以 4-byte 為單位對齊, branch offset 需要轉換為 byte address。這裡也能支援負 offset, 因此可實現 backward branch 與 loop。

- 2) Jump Address Logic

用於處理 j 指令。Jump instruction 中的 address field 為 26 bits, 因此需要左移 2 bits 形成 28-bit 位址, 再與 Input_Addr + 4 的最高 4 bits 組合成 32-bit jump address。

$\text{jump_addr} = \{(\text{PC}+4)[31:28], \text{instruction}[25:0], 2'b00\}$

當 Jump = 1 時, 透過 MUX, 下一個 instruction address 會選擇 jump_addr, 使 CPU 跳過中間指令並執行指定位置的 instruction。

3) ALU.v

新增對 beq 的支援。ALU 會對兩個 register 判斷是否相等, 因此執行 subtraction, 若結果為 0, 則代表兩個 register 相等, ALU 會輸出 Zero = 1。此 Zero signal 會與 Branch control signal 透過 AND 閘決定 branch 是否成立。

4) Control.v & ALU_Control.v

加入 Branch 與 Jump。當 opcode 為 beq 時, Control 會啟用 Branch, 並設定 ALU 進行 subtraction; 當 opcode 為 j 時, Control 會啟用 Jump, 使下一個 address 直接由 jump address logic 決定。

● Datapath & Controlpath

1) Datapath

新增 branch address logic、jump address logic, 以及 next address select mux。一般指令仍使用 Input_Addr + 4 作為下一個 instruction address; 當 beq 成立時, 則選擇 branch_addr; 當執行 j 指令時, 則選擇 jump_addr。

在 beq 指令中, CPU 會從 RF 讀出 Rs_data 與 Rt_data, 並送入 ALU 執行 subtraction。若 ALU result 為 0, Zero 會被設為 1, 表示兩個 register 相等。此時若 Branch = 1, next address mux 會選擇 branch target address, 使程式跳到指定位置。

在 j 指令中, CPU 不需要比較 register, 也不需要存取 DM 或寫回 RF, 而是直接根據 instruction 中的後 26-bit 並產生 jump_addr, 將其作為下一個 Output_Addr。

2) Controlpath

Control path 由 Control、ALU_Control、ALU 的 Zero signal, 以及 next address select logic 共同完成。Control 根據 opcode 判斷目前指令是否為 beq 或 j, 並產生 Branch、Jump、ALU_op 等控制訊號。ALU_Control 則根據 ALU_op 決定 ALU 是否執行 subtraction。

Behavioral Simulation

1. Input Pattern: IM.dat and DM.dat

Instructions 以及對應動作, 剩餘 instructions 全部補 00 00 00 00:

1	24 01 00 02	// addiu \$1, \$0, 2	; R1 = 2, 作為 loop counter
2	34 02 FF FF	// ori \$2, \$0, 0xFFFF	; R2 = 0000FFFF, 測 zero extension
3	24 03 FF FF	// addiu \$3, \$0, -1	; R3 = FFFFFFFF, 測 sign extension
4	10 43 00 01	// beq \$2, \$3, 1	; not taken, 因 0000FFFF != FFFFFFFF
5	AC 02 00 00	// sw \$2, 0(\$0)	; DM[0] = 0000FFFF
6	10 21 00 01	// beq \$1, \$1, 1	; taken, 跳過下一條
7	AC 03 00 04	// sw \$3, 4(\$0)	; skipped, 不應執行
8	8C 04 00 00	// lw \$4, 0(\$0)	; R4 = DM[0] = 0000FFFF
9	00 81 28 21	// addu \$5, \$4, \$1	; R5 = 0000FFFF + 2 = 00010001
10	AC 05 00 04	// sw \$5, 4(\$0)	; DM[4] = 00010001
11	08 00 00 0D	// j 0x0000000D	; jump to instruction 14
12	24 06 00 CC	// addiu \$6, \$0, 0x00CC	; skipped
13	AC 06 00 08	// sw \$6, 8(\$0)	; skipped
14	24 21 FF FF	// addiu \$1, \$1, -1	; loop: R1 = R1 - 1
15	10 20 00 01	// beq \$1, \$0, 1	; if R1 == 0, exit loop
16	10 00 FF FD	// beq \$0, \$0, -3	; branch back to loop
17	AC 01 00 08	// sw \$1, 8(\$0)	; DM[8] = 00000000
18	AC 03 00 0C	// sw \$3, 12(\$0)	; DM[12] = FFFFFFFF

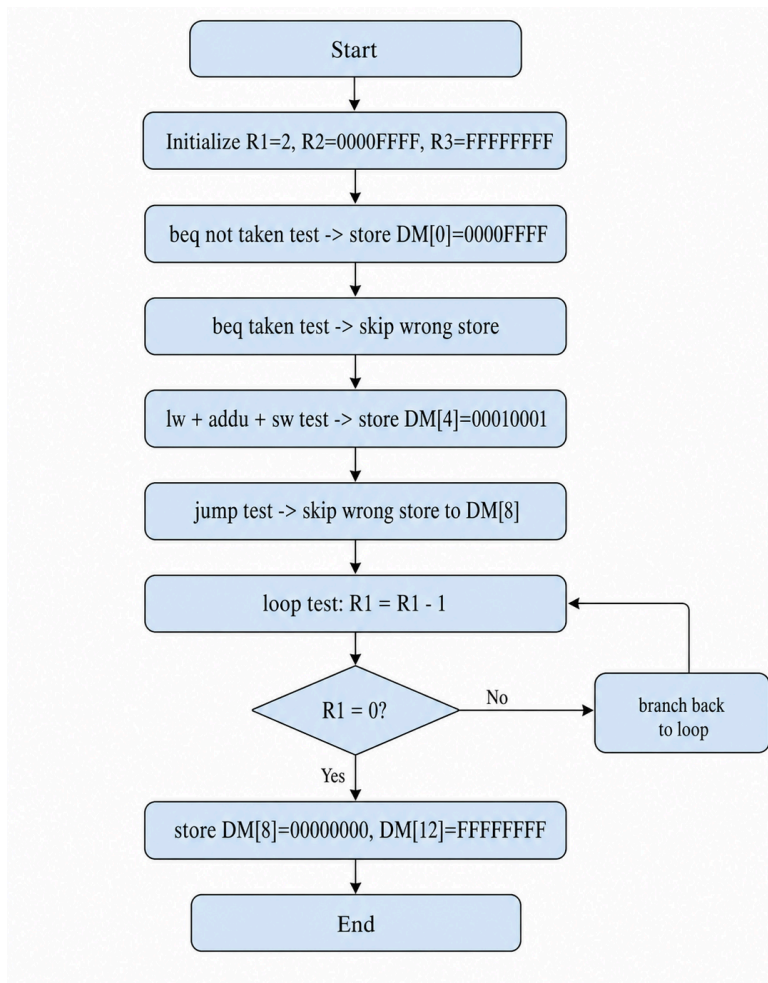
Data Memory 初始值, 全部皆為00

1	00
2	00
3	00
4	00
5	00
6	00
7	00
8	00
9	00
10	00

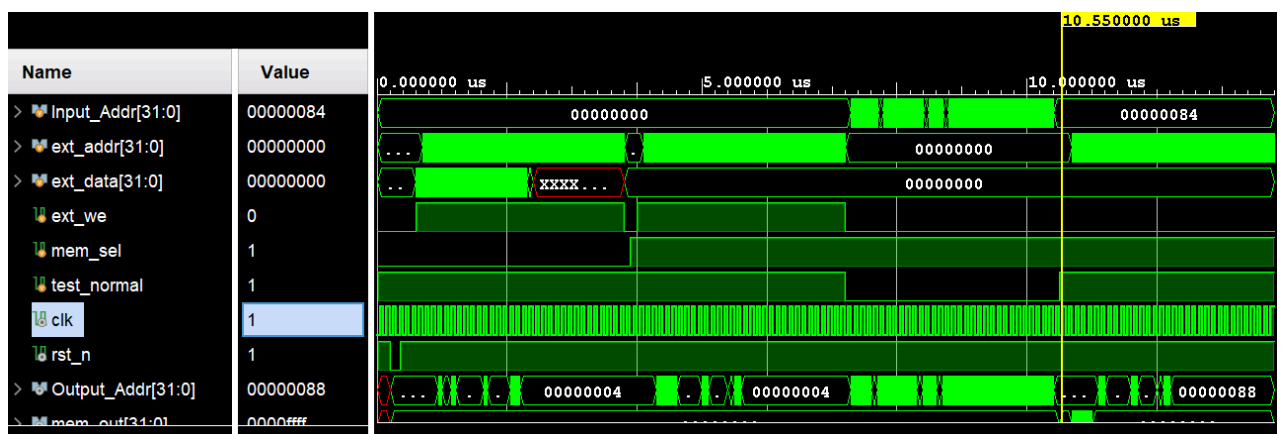
2. Testbench Explanation

在這個testbench中, 我首先利用 ori 與 addiu -1 分別測試 zero extension 與 sign extension, 再以 beq 測試 taken 與 not taken 兩種情況。接著透過 lw、sw 驗證 Data Memory 讀寫, 並加入 R-format addu 測試 ALU 運算。最後使用 j 與負 offset beq 建立 loop, 確認 jump 與 backward branch 可正確更新 Output_Addr。共有 17 個有效 instruction cycles 會產生主要測試結果。之後由於剩餘 instruction memory 皆補 00 00 00 00。

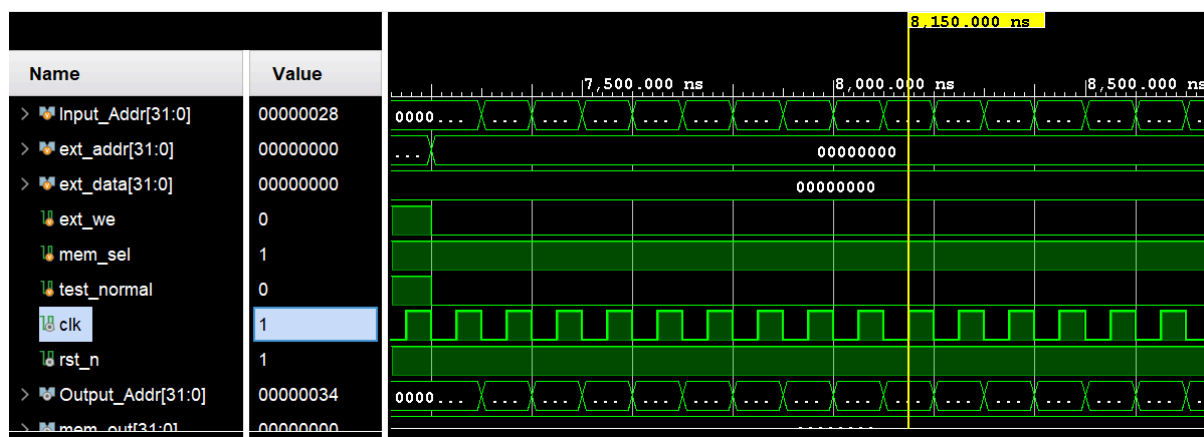
3. Flowchart



4. Waveform



以此圖為例，目前 Input_Addr = 32'h00000028，正在執行 j 0x0000000D。一般情況下下一個位址會是 PC+4 = 32'h0000002C，但由於 Jump = 1，next address mux 會選擇 jump target。jump target = {(PC+4)[31:28], instruction[25:0], 2'b00} = 32'h00000034，因此 Output_Addr 變為 32'h00000034。



5. Vivado tcl console

可以看到最終 Data Memory 中數值符合預期

```
run all
IM loaded
DM Loading...
DM loaded

--- Start to Run Instructions in IM ---
--- Instructions in IM are done ---

--- [Data Memory Results] ---
DM[ 0] : 0000ffff
DM[ 4] : 00010001
DM[ 8] : 00000000
DM[12] : ffffffff
DM[16] : 00000000

Simulation Completed. By IEESLab

  /\_/\  /\_/\  /\_/\  Congratulations !
  ( o.o ) ( -.- ) ( ^.^ ) Please Check <<DM.out>>
  > ^ <  > ^ <  > ^ <  Wishing you continued success and happiness. By IEESLab

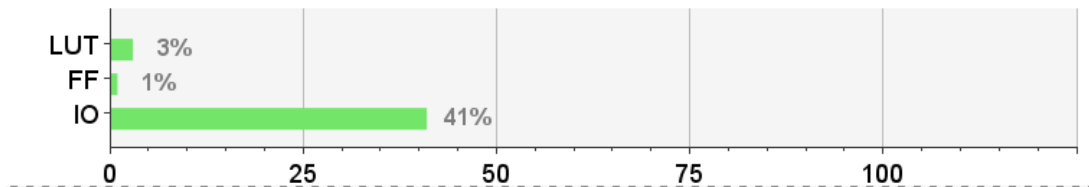
$finish called at time : 13800 ns : File "C:/Vivados_Verilog_project/SimpleCPU/TESTBED/tb_SimpleCPU.v" Line 208
```

Synthesis Result

Synthesis 結果顯示，本設計已成功通過 timing constraint, Setup、Hold 與 Pulse Width 皆無 violation, 代表電路在目前時脈限制下仍有足夠 timing margin。

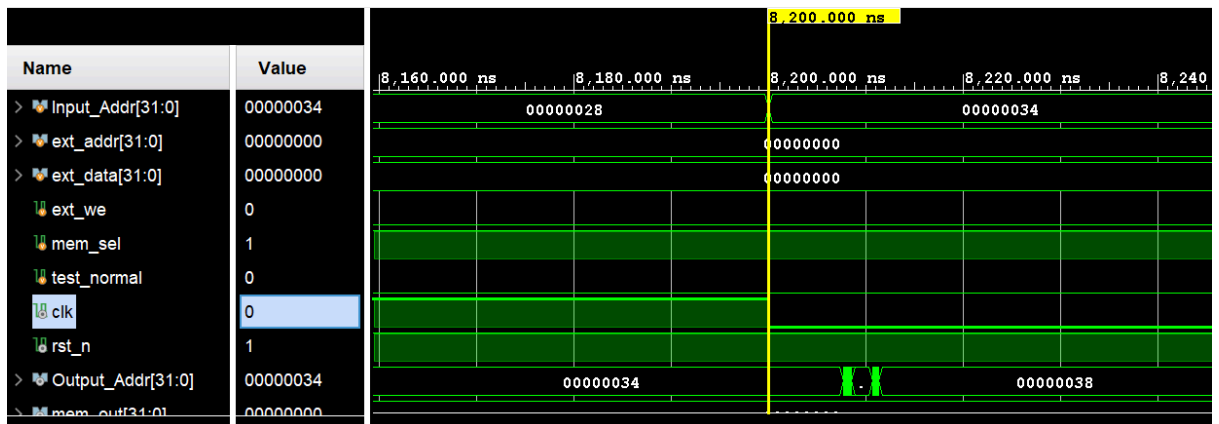
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 41.065 ns	Worst Hold Slack (WHS): 0.846 ns	Worst Pulse Width Slack (WPWS): 19.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 9280	Total Number of Endpoints: 9280	Total Number of Endpoints: 3073
All user specified timing constraints are met.		

Resource	Utilization	Available	Utilization %
LUT	4612	134600	3.43
FF	3072	269200	1.14
IO	165	400	41.25



Post-Synthesis Timing Simulation

1. Waveform



可以透過波形圖觀察到, Output_Addr 的輸出時間相較於 behavioral simulation 有些延後。這是因為在 post-synthesis simulation 中, 電路已經經過合成, adder 與 next-address mux 等組合邏輯會具有實際的 propagation delay。因此, 當 Input_Addr 改變時, Output_Addr 可能會先短暫出現不穩定的暫態值。待組合邏輯穩定後, Output_Addr 會回到正確的下一個 PC 值, 因此此現象屬於合成後模擬中的正常延遲現象。

2. Vivado tcl console

可以看到最終 Data Memory 中數值一樣符合預期並完整跑完

```

run all
IM loaded
DM Loading...
DM loaded

--- Start to Run Instructions in IM ---
--- Instructions in IM are done ---

--- [Data Memory Results] ---
DM[ 0] : 0000ffff
DM[ 4] : 00010001
DM[ 8] : 00000000
DM[12] : ffffffff
DM[16] : 00000000
DM[20] : 00000000

Simulation Completed. By IEESLab

/\_/\  /\_/\  /\_/\  Congratulations !
( o.o ) ( -.- ) ( ^_^ ) Please Check <<DM.out>>
> ^ <  > ^ <  > ^ <  Wishing you continued success and happiness. By IEESLab

```

Conclusion

這次作業由 R_FormatCPU、I_FormatCPU 到 SimpleCPU 逐步完成 single-cycle CPU 的設計。Part I 先實作基本 R-format datapath, 包含 instruction fetch、register read、ALU operation 與 register 的 write-back。Part II 在此基礎上加入 immediate extension、Data Memory 與 write-back mux, 使 CPU 能支援 addiu、ori、lw、sw 等 I-format 指令。Part III 則進一步加入 branch 與 jump address logic, 使 CPU 能執行 beq、j 與簡單 loop。透過 behavioral simulation、test pattern 驗證與 synthesis 結果, 可以確認三個 CPU 皆能依照預期完成指令執行與資料寫回, 並符合 single-cycle CPU 的設計目標。

心得

這次的作業相對複雜, 我認為最困難的部分在於各模組之間的接線, 以及將整體 datapath 與 control path 畫成完整的硬體架構圖。因為每個控制訊號都會影響資料的流向, 只要其中一條線接錯, 就可能導致整個執行結果錯誤。不過也因為經過這樣的設計與除錯過程, 讓我能夠重新整理上課所學的概念, 並將 instruction fetch、decode、ALU operation、memory access 與 write-back 等流程實際應用在本次作業中。整體而言, 這次作業幫助我更清楚理解 Single CLK cycle CPU datapath 與 control signal 之間的關係, 使我更加期待之後 pipeline 架構的部分。